

Contents

1	kNN for classification	2
2	Min-max normalisation	2
3	Training (and testing) the kNN model	3
4	German credit dataset	4
5	Training (and evaluating) the model	5
5.1	Why <code>set.seed()</code> ?	7
6	Evaluating the final model	7
7	Extra: criticism of this training and testing process	7
8	Comparison with logistic regression	7
8.1	Extra: choice of decision threshold	8
9	Extra: limitations of kNN classification	8
10	Extra: kNN classification with <code>caret</code> package	8

1 kNN for classification

Several weeks ago, we used kNN for a regression task. This week, we apply the kNN algorithm to classification. The ideas are largely the same, just that we are predicting a category instead of a number. So, instead of taking an average of the k nearest neighbours outcome variables (as in regression), we will instead have a majority vote of the k nearest neighbours on what category it belongs to (for classification).

In R, we will use the `knn()` function from the `class` package for our kNN classification task. We will follow the tutorial code here, focusing again on the BreastCancer dataset in the `mlbench` package we used last week. We will prepare the dataset in the same manner, and focus on the same predictors.

```
data("BreastCancer", package="mlbench")
bc <- na.omit(BreastCancer)

bc[,2:4] <- sapply(bc[,2:4], as.numeric)
bc <- bc %>% mutate(y = factor(ifelse(Class=="malignant", 1, 0))) %>%
  select(Cl.thickness:Cell.shape, y)
str(bc)
```

```
## 'data.frame':    683 obs. of  4 variables:
##  $ Cl.thickness: num  5 5 3 6 4 8 1 2 2 4 ...
##  $ Cell.size   : num  1 4 1 8 1 10 1 1 1 2 ...
##  $ Cell.shape  : num  1 4 1 8 1 10 1 2 1 1 ...
##  $ y           : Factor w/ 2 levels "0","1": 1 1 1 1 1 2 1 1 1 1 ...
## - attr(*, "na.action")= 'omit' Named int [1:16] 24 41 140 146 159 165 236 25
## ..- attr(*, "names")= chr [1:16] "24" "41" "140" "146" ...
```

Unlike the `train()` function from `caret` which pre-processes our data (by centering and scaling), the `knn()` function does not pre-process the data. We thus need to manually pre-process the data for training, to account for any different scales between the predictors when using the kNN algorithm. As a refresher, the scaling done in the `train()` function is to *center* the mean of each predictor to 0, and to *scale* the standard deviation of each predictor to 1. How this is done mathematically is outside the scope of this course.

2 Min-max normalisation

We will perform a simple *normalisation* to have all the predictors on the same scale, which is to compress *every value in each predictor* to be between 0 and 1. This normalisation is called **min-max scaling** or **min-max normalisation**.

As an example, say that our vector of a predictor is `c(1, 2, 1, 3, 5)`. Performing min-max normalisation on this vector will move the minimum value to 0, the maximum value to 1, and every

value in between will be scaled between them. In this vector, 1 becomes 0, 5 becomes 1, and 2 and 3 will be scaled to be between 1 and 5. For example, 2 is 25% of the way from 1 to 5, and so will become 0.25. The resulting vector is `c(0, 0.25, 0, 0.5, 1)`. Mathematically, the transformation is:

$$x_{\text{scaled}} = \frac{x - \min(x)}{\max(x) - \min(x)}$$

Since we want to scale every predictor to have a min value of 0 and max value of 1, we have to apply this min-max scaling to every predictor. We define a function that applies this min-max normalisation, then *apply* it to every predictor with `sapply()`. Looking at the summary of the dataset, you can see the effects of the normalisation.

```
nor <- function(x) { (x - min(x)) / (max(x) - min(x)) }
bc[1:3] <- sapply(bc[1:3], nor)
summary(bc)
```

```
##   Cl.thickness      Cell.size      Cell.shape      y
##   Min.      :0.0000    Min.      :0.0000    Min.      :0.0000    0:444
##   1st Qu.:0.1111    1st Qu.:0.0000    1st Qu.:0.0000    1:239
##   Median :0.3333    Median :0.0000    Median :0.0000
##   Mean   :0.3825    Mean   :0.2390    Mean   :0.2461
##   3rd Qu.:0.5556    3rd Qu.:0.4444    3rd Qu.:0.4444
##   Max.    :1.0000    Max.    :1.0000    Max.    :1.0000
```

For this dataset, all the predictors we wished to use were already on a scale of 1 to 10, so the normalisation would not have had a big effect (if any) on the model. The train and test datasets are generated in the same way.

```
set.seed(100)
train.idx <- sample(nrow(bc), size = nrow(bc)*0.8)
train.data <- bc[train.idx, ]
test.data <- bc[-train.idx, ]
```

3 Training (and testing) the kNN model

We will use the `knn()` function from the `class` package. Again, this differs greatly from the `train()` function in `caret` in three main ways:

1. the predictors and outcome variables are passed to the function **separately**. Columns 1 to 3 include the *predictors only*, and the outcome variable `y` is passed in to the `cl` parameter. The `cl` parameter also only accepts *vectors*, which we get with the `$` notation.
2. the train and test datasets are passed in to the function at the same time. The model is trained on the train dataset, and then immediately predicts the test dataset.

- the value of k to be tested must be passed in. This makes testing different values of k slightly convoluted.

Columns 1 to 3 of the dataset contain our predictors, and we choose to test *only* $k = 2$. With this value of k , we achieve an accuracy of 96.35%.

```
library(class)
set.seed(101)
yhat <- knn(train.data[1:3], test.data[1:3], cl=train.data$y, k=2)
mean(yhat == test.data$y) # compute accuracy
```

```
## [1] 0.9635036
```

4 German credit dataset

The tutorial notes continue on to focus on a different dataset, on German credit applications. The data is stored in a .csv file and contains 1000 observations and 21 variables, many of which are encoded (for secrecy, anonymity, etc.). The variable we will focus on predicted is Default, indicating whether that particular loan was defaulted on or not. If we can predict if a loan will default given some predictors, then we can make a more informed decision on whether to approve the loan or not.

We will focus on the numeric variables in this dataset, and scale them using the `nor()` function we defined above.

```
gc <- read.csv("germancredit.csv", header = TRUE, sep=",")

numvars <- sapply(gc, is.numeric)
gc1 <- gc[numvars]
gc1 <- data.frame(sapply(gc1, nor))
gc1$Default <- factor(gc1$Default)
str(gc1)
```

```
## 'data.frame':    1000 obs. of  8 variables:
## $ Default      : Factor w/ 2 levels "0","1": 1 2 1 1 2 1 1 1 1 2 ...
## $ duration     : num  0.0294 0.6471 0.1176 0.5588 0.2941 ...
## $ amount       : num  0.0506 0.3137 0.1016 0.4199 0.2542 ...
## $ installment: num   1 0.333 0.333 0.333 0.667 ...
## $ residence    : num   1 0.333 0.667 1 1 ...
## $ age          : num  0.8571 0.0536 0.5357 0.4643 0.6071 ...
## $ cards        : num  0.333 0 0 0 0.333 ...
## $ liable       : num  0 0 1 1 1 1 0 0 0 0 ...
```

5 Training (and evaluating) the model

The dataset is split into a training and testing dataset as usual.

```
set.seed(100)
train.idx <- sample(nrow(gc1), nrow(gc1)*0.8)
train.data <- gc1[train.idx,]
test.data <- gc1[-train.idx,]
```

Then, using the `knn()` function from `class`, we will have to pass it different values of k to test. First, we create a vector `accuracy` to store the results of each kNN model, initialising it to have all 0s. Then, we iterate over values from 1 to 30, training and evaluating a kNN model with different k values, recording the accuracy on the test set.

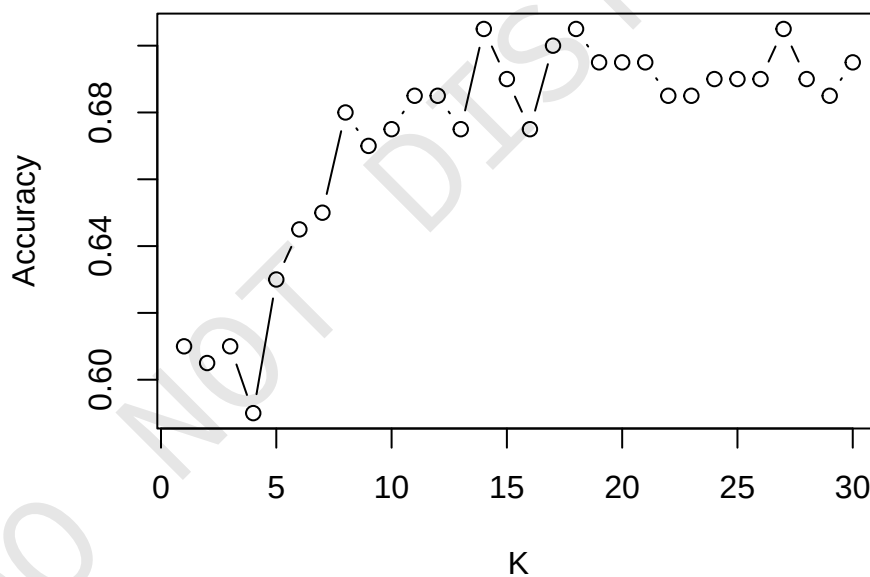
The `cat()` is short for **concat**enation, i.e. joining strings. It just joins together what you pass it, and prints it, functioning extremely similarly to `print()` in Python.

```
accuracy <- rep(0, 30)
for (i in 1:30) {
  set.seed(101)
  knn.i <- knn(train.data[2:4], test.data[2:4], cl=train.data$Default, k=i)
  accuracy[i] <- mean(knn.i == test.data$Default)
  cat("k =", i, "accuracy =", accuracy[i], "\n")
}
```

```
## k = 1 accuracy = 0.61
## k = 2 accuracy = 0.605
## k = 3 accuracy = 0.61
## k = 4 accuracy = 0.59
## k = 5 accuracy = 0.63
## k = 6 accuracy = 0.645
## k = 7 accuracy = 0.65
## k = 8 accuracy = 0.68
## k = 9 accuracy = 0.67
## k = 10 accuracy = 0.675
## k = 11 accuracy = 0.685
## k = 12 accuracy = 0.685
## k = 13 accuracy = 0.675
## k = 14 accuracy = 0.705
## k = 15 accuracy = 0.69
## k = 16 accuracy = 0.675
## k = 17 accuracy = 0.7
## k = 18 accuracy = 0.705
## k = 19 accuracy = 0.695
```

```
## k = 20 accuracy = 0.695
## k = 21 accuracy = 0.695
## k = 22 accuracy = 0.685
## k = 23 accuracy = 0.685
## k = 24 accuracy = 0.69
## k = 25 accuracy = 0.69
## k = 26 accuracy = 0.69
## k = 27 accuracy = 0.705
## k = 28 accuracy = 0.69
## k = 29 accuracy = 0.685
## k = 30 accuracy = 0.695
```

```
plot(accuracy, type="b", xlab="K", ylab="Accuracy")
```



```
which.max(accuracy)
```

```
## [1] 14
```

The accuracy plot indicates three maxima of 70.5% at $k = 14$, $k = 18$, and $k = 27$. Using `which.max` lets us see the *first index* which the maximum value occurs at. In this case, the i th index of accuracy represents the accuracy of the kNN model with $k = i$, so the index is equal to the k value tested. In practice, we will opt for the model with the smallest value of k , i.e. $k = 14$, since it is simpler as it takes into account less data points.

5.1 Why `set.seed()`?

You may see that `set.seed()` is included in this code, even though the training/evaluation does not seem to include any random processes. However, when there are **ties** in the decision process (when there is an even number of both classes among the k nearest neighbours), the final class is determined randomly. This occurs more often for even values of k . For odd numbers of k , this may still occur when there are ties for the k th nearest neighbour, as these tied neighbours are all included in the vote for the model's predicted class.

6 Evaluating the final model

We will test the kNN model with $k = 14$ and see its confusion matrix.

```
set.seed(101)
yhat.knn <- knn(train.data[2:4], test.data[2:4], cl=train.data$Default,
  ↪ k=14)
mean(yhat.knn == test.data$Default)
```

```
## [1] 0.705
```

```
table(predicted=yhat.knn, observed=test.data$Default)
```

```
##      observed
## predicted    0    1
##           0 127  48
##           1  11  14
```

7 Extra: criticism of this training and testing process

In this training process, we used the test data to determine the final choice of our parameter k in the model. In general, this is bad practice, as this biases your model towards the test data, and so the metrics that the model achieves are not as generalisable to future new observations. Instead, the training data should be split up into two sets, similar to cross-validation.

8 Comparison with logistic regression

A logistic model is trained on the same predictors and evaluated. With a decision threshold of $p = 0.5$, you can see that the false negative rate is much higher at 93.5%. One thing to note about this model: because the data are scaled differently, the **real-world interpretation** of the coefficients is extremely different. **A unit change in the predictor is now the whole range.**

```
m.logistic <- glm(Default ~ ., data = train.data[1:4], family = "binomial")
yhat.p <- predict(m.logistic, test.data, type = "response")
yhat.logistic <- factor(ifelse(yhat.p > 0.5, 1, 0))
mean(yhat.logistic == test.data$Default)
```

```
## [1] 0.68
```

```
table(predicted=yhat.logistic, observed=test.data$Default)
```

```
##           observed
## predicted    0    1
##           0 132  58
##           1   6   4
```

8.1 Extra: choice of decision threshold

Again, the choice of a decision threshold has clear implications in this example. As the bank giving out the loan, what decision threshold would you set: $p = 0.7$ or $p = 0.3$? What do they lead to, in your model, and in the real world?

9 Extra: limitations of kNN classification

kNN classifications faces one significant limitation, which is when the classes are *imbalanced*. For example, in this credit dataset, are the outcome classes extremely skewed towards defaulting? Looking at the training dataset, we can see that there are significantly more observations that do not default, about 2.36x more. This skew in the dataset already leads the model's predictions towards 'no default', though not significantly so. You may see datasets with extremely skewed classes, e.g. 10:1 ratio.

```
summary(train.data$Default)
```

```
##    0    1
## 562 238
```

10 Extra: kNN classification with caret package

kNN actually works perfectly well with the `train()` function from `caret`. You can specify the exact same options, including the values of k to test, and how the data should be pre-processed. This separates the training phase from the final model evaluation, and so is much more robust.

You may use this, but figure out the details on your own. Note that the `preProcess` argument may not be needed, since you may have already scaled the data.

